
TP2 — Bibliothèque Upload et Lecture Audio

Mission 2 — Bibliothèque paginée

Déjà fonctionnelle dans le projet fourni (vérifiée, non modifiée) :
`TrackService.list(page, limit)` transmet bien `page` et `limit` au serveur, et les Signals `tracks`, `page`, `pages`, `loading` existaient déjà. Seul le Signal `error` manquait et a été ajouté.

Mission 3 — Upload et lecture audio

Flux d'upload
1. Choix du fichier : `tracks-page.html` (`(change)="choose($event)"`) → `tracks-page.ts`, méthode `choose()`.
2. Construction du `FormData` : `track.service.ts`, méthode `upload()` (`body.append('audio', file); body.append('title', title)`).
3. Appel HTTP : `track.service.ts`, `this.http.post<Track>('/api/tracks', body)`.

Flux de lecture
1. Récupération du `Blob` : `track.service.ts`, méthode `audio()` (`responseType: 'blob'`).
2. Création de l'`ObjectURL` : `tracks-page.ts`, méthode `play()` (`URL.createObjectURL(blob)`).
3. Affectation au lecteur : `tracks-page.html`, `<audio [src]="audioUrl()">`.
4. Révocation : avant chaque nouvelle lecture (`play()`) et à la destruction du composant (`ngOnDestroy()`).

Intercepteur JWT
`auth.interceptor.ts` ajoute `Authorization: Bearer <token>` à toute requête passée par `HttpClient`, y compris `GET /api/tracks/:id/audio`. Une URL placée directement dans `src` serait chargée par le navigateur en dehors d'Angular, donc sans passer par l'intercepteur : l'en-tête ne serait jamais ajouté. C'est pourquoi le fichier est récupéré en `Blob` via `HttpClient`, puis transformé en `ObjectURL` locale.

Validation frontend vs backend
La validation ajoutée dans `choose()` (type de fichier, taille max 25 Mo) améliore l'expérience utilisateur en affichant l'erreur immédiatement, sans requête réseau. Elle ne remplace jamais la validation backend, car le frontend peut être contourné (DevTools, Postman, curl) : seule la vérification côté serveur est une garantie fiable.

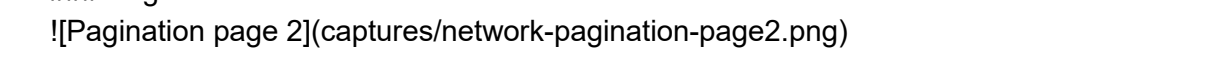
Éléments d'interface ajoutés
Signals `uploading`, `uploadError`, `uploadSuccess` : bouton désactivé pendant

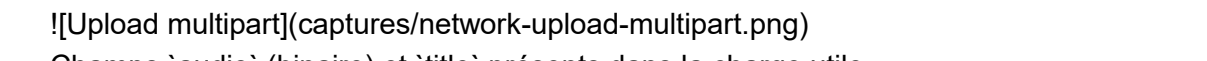
l'envoi (anti double-soumission), message de succès, message d'erreur serveur, formulaire vidé et première page rechargée après succès. Cards présentées avec titre, nom original, format, taille (Mo), date, et bouton de lecture.

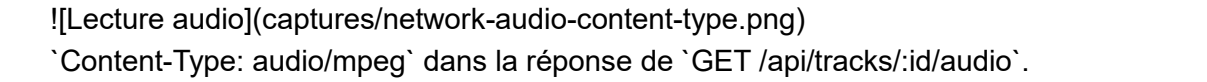
Questions mémoire, buffering, streaming

- Le backend peut servir le fichier par flux depuis le disque plutôt que de le charger entièrement en mémoire avant de répondre.
- Avec `responseType: 'blob'`, le composant reçoit le fichier seulement une fois que `HttpClient` a reçu l'intégralité de la réponse (l'Observable émet `next()` à la fin du téléchargement).
- Non : `GET /api/tracks` ne renvoie que les métadonnées. Le fichier audio n'est téléchargé qu'au clic sur ▶ , via `service.audio(track.id)`.
- Avec 100 éléments `<audio src="...">` directs, les requêtes ne passeraient pas par l'intercepteur JWT (donc échoueraient en 401), et le navigateur pourrait précharger plusieurs fichiers automatiquement selon son réglage `preload`.
- `URL.createObjectURL()` garde le `Blob` en mémoire tant qu'il n'est pas révoqué ; sans `URL.revokeObjectURL()`, la mémoire s'accumule à chaque nouvelle piste écoutée.

Checkpoint Network

Pagination

`GET /api/tracks?page=2&limit=5` → confirmé dans l'URL de requête.

Upload multipart

Champs `audio` (binaire) et `title` présents dans la charge utile.

Lecture audio authentifiée

`Content-Type: audio/mpeg` dans la réponse de `GET /api/tracks/:id/audio`.